PROJECT REPORT

Project Name: DISK SCHEDULING SYSTEM

Session:2023-2027

(Reg. No. 2023-UOK-03604, 2023-UOK-03612, 2023-UOK-03676)

Submitted To:

Mr. Mehtab Mushtaq

Subject: OPERATING SYSTEM

Submitted By:

Hammad Riaz (04), Manzar Shamim (12), Hassan Razzaq (76)

Date of Submission: February 13, 2026

Version 2.0

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF COMPUTING AND ENGINEERING

UNIVERSITY OF KOTLI AZAD JAMMU AND KASHMIR

DISK SCHEDULING SYSTEM

(Reg. No. 2023-UOK-03604, 2023-UOK-03612, 2023-UOK-03676)

Subject: Operating System

Session:2023-2027

Submitted By:

Hammad Riaz (04), Manzar Shamim (12), Hassan Razzaq (76)

Date of Submission: February 13, 2026

DEPARTMENT OF COMPUTER SCIENCE
FACULTY OF COMPUTING AND ENGINEERING
UNIVERSITY OF KOTLI AZAD JAMMU AND KASHMIR

Contents

1. SYSTEM REQUIREMENTS 4
1.1-Functional Requirements 4
1.1.1-User Input Handling 4
1.1.2-Algorithm Selection 4
1.1.3-Request Processing 4
1.1.4-Head Movement Calculation 4
1.1.5-Output Display 4
1.1.6-Algorithm Comparison 4
1.1.7-Error Handling 5
1.1.8-Reusability 5
1.1.9-Extensibility 5
1.1.10-Termination 5
1.2-Non-Functional Requirements 5
2.COMPONENT DIAGRAM 6
2.1-User Interface & Simulation Controller Architecture 6
2.2-Simulation Controller and IS Scheduler Architecture 7
2.3-Simulation Controller, IS Scheduler and Comparison Engine Architecture 8
2.4-Is Scheduler and Disk Scheduling Algorithms 9
3.FINAL COMPONENT DIAGRAM OF DISK SCHEDULING SYSTEM 10

1. SYSTEM REQUIREMENTS

This section outlines the functional and non-functional requirements of the Disk Scheduling Simulator. It explains the core functionalities that the system must provide and the operational constraints it must follow.

1.1-Functional Requirements

1.1.1-User Input Handling

- Disk request queue (list of track numbers)
- Initial disk head position
- Disk size (optional, for SCAN-type algorithms)
- Direction of head movement (left or right)

1.1.2-Algorithm Selection

- FCFS
- SSTF
- SCAN
- C-SCAN
- LOOK
- C-LOOK

1.1.3-Request Processing

- The system shall process disk requests according to the selected scheduling algorithm.
- The system shall determine the order in which disk requests are serviced.

1.1.4-Head Movement Calculation

- The system shall calculate the total disk head movement for each algorithm.
- The system shall calculate seek distance between consecutive disk requests.

1.1.5-Output Display

The system shall display:

- The order of disk head movement
- Total head movement
- Individuals seek operations
- The output shall be shown in a clear and readable format.

1.1.6-Algorithm Comparison

- The system shall allow comparison of multiple disk scheduling algorithms.
- The system shall show which algorithm gives minimum head movement for given input.

1.1.7-Error Handling

- The system shall validate user input.
- Invalid track numbers
- Empty request queue
- Head position outside disk range

1.1.8-Reusability

- The system shall allow repeated execution with different inputs without restarting the program.

1.1.9-Extensibility

- The system shall support adding new disk scheduling algorithms in the future.

1.1.10-Termination

- The system shall allow the user to exit the program safely.

1.2-Non-Functional Requirements

- The system must be user-friendly and easy to operate.
- The system must run on Windows and Linux platforms.
- The system must provide accurate and reliable results.
- The system must handle up to 50 disk requests.
- The system must be easy to maintain and extend.

2.COMPONENT DIAGRAM

2.1-User Interface & Simulation Controller Architecture

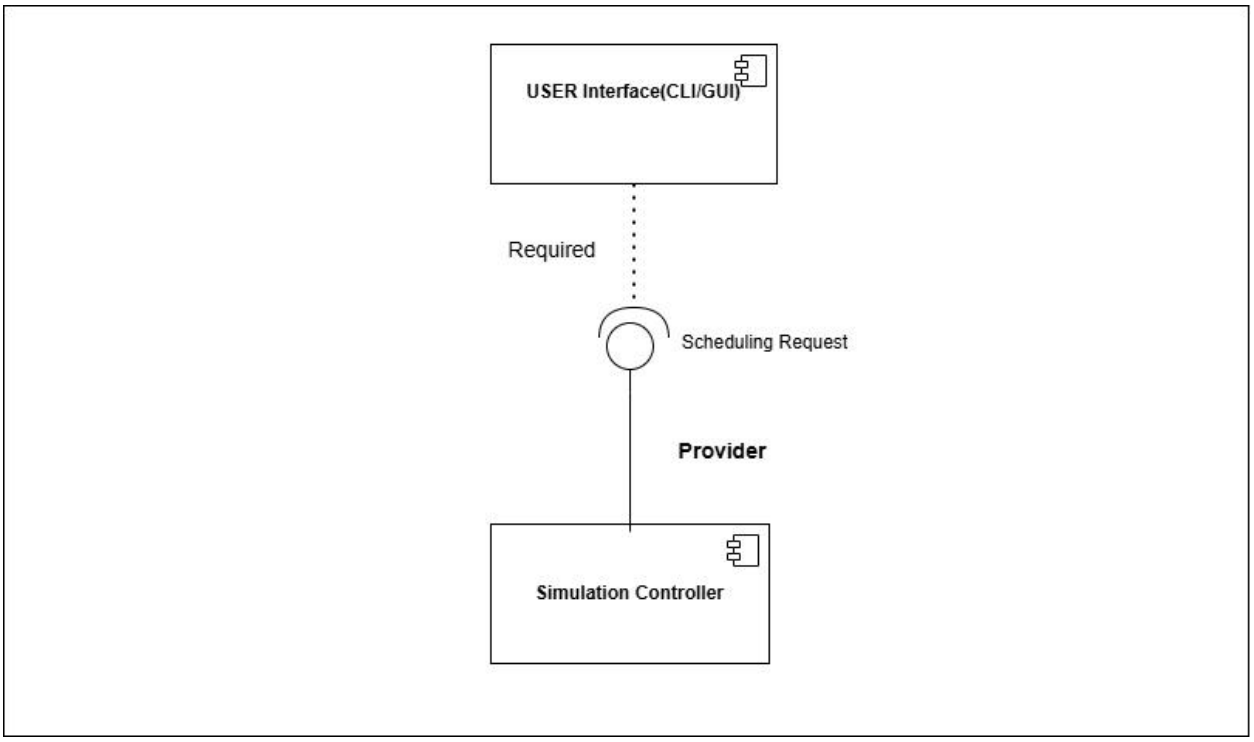

Figure 2.1 User Interface & Simulation Controller

Explanation:

In this component diagram the main components are User Interface (CLI/GUI) and the Simulation Controller, connected through an interface. The User Interface is responsible for taking input from the user, such as disk requests, head position, and selected algorithm, and displaying the results. It requires processing services from the Simulation Controller.

The Simulation Controller performs the core logic of the system. It executes the selected disk scheduling algorithm, calculates the head movement and total seek time, and sends the results back to the User Interface. The interface between them defines how the User Interface requests services and how the Controller provides them within the system.

2.2-Simulation Controller and IS Scheduler Architecture

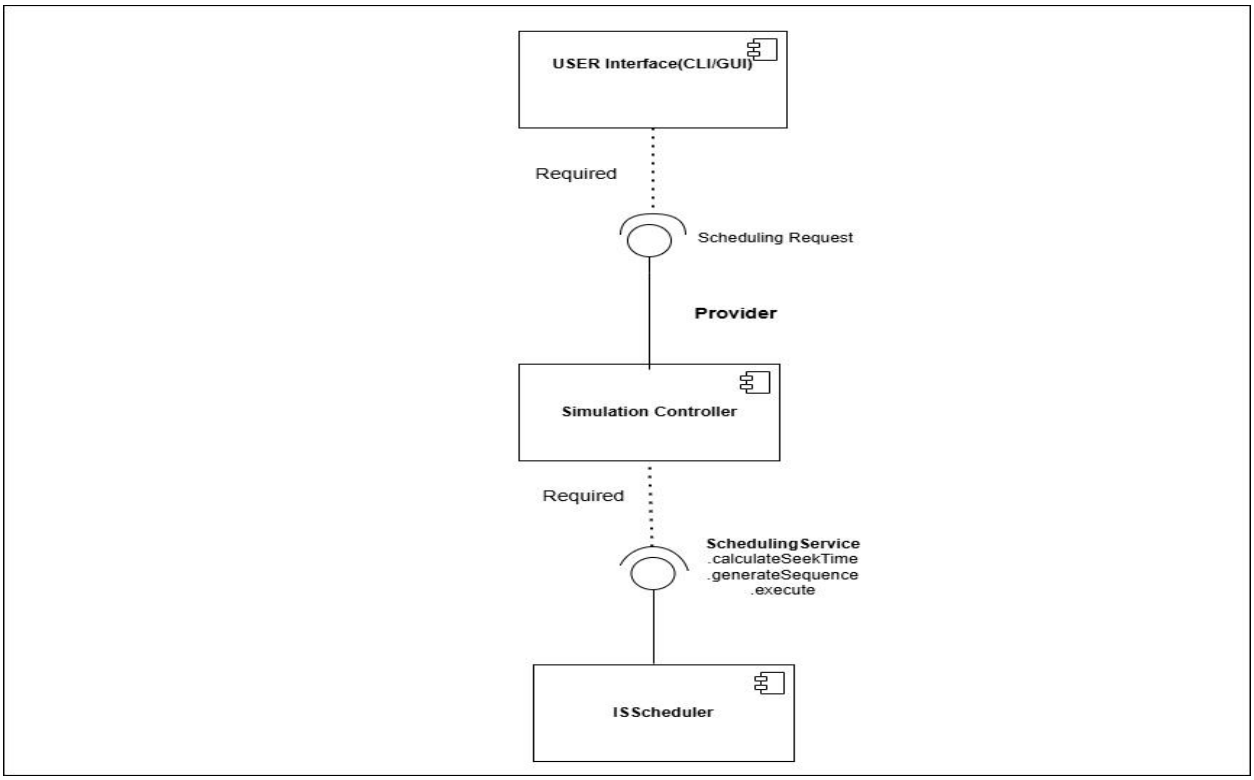

Figure: 2.2 Simulation Controller and IS Scheduler

Explanation:

This part of the component diagram represents the core coordination and service layer of the Disk Scheduling Simulator. The Simulation Controller acts as the central controlling component that manages the overall scheduling process. It receives scheduling requests from the User Interface and forwards them to the Scheduling Service for execution.

The Scheduling Service interface defines the main operations required for disk scheduling, such as calculating total seek time, generating the seek sequence, and executing the selected algorithm. This interface ensures that all scheduling operations follow a consistent structure.

The IS Scheduler component acts as a common abstraction for all disk scheduling algorithms. It provides a unified interface that must be implemented by each specific scheduling algorithm. By using this interface-based design, the system achieves loose coupling and high modularity.

2.3-Simulation Controller, IS Scheduler and Comparison Engine Architecture

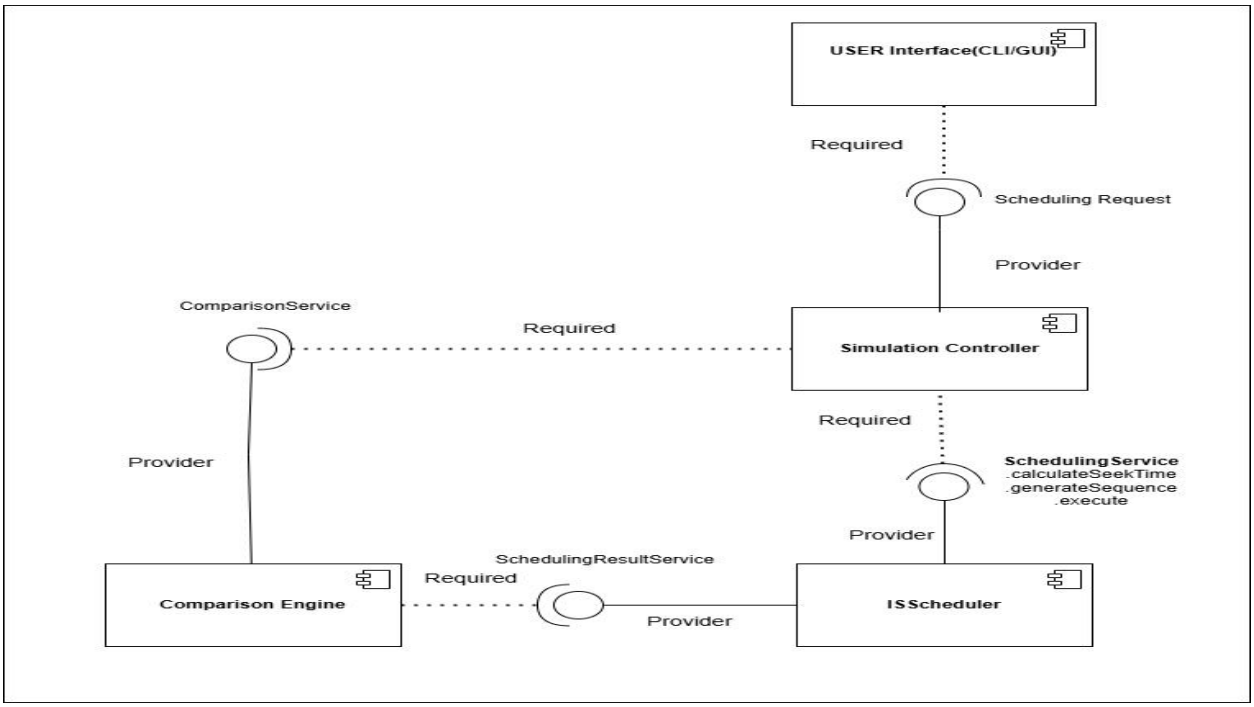

Figure: 2.3 Simulation Controller, IS Scheduler and Comparison Engine

Explanation:

This part of the component diagram represents the central processing and result comparison layer of the Disk Scheduling Simulator. The Simulation Controller acts as the main coordinating component of the system. It receives scheduling requests from the User Interface and manages the execution flow of the selected disk scheduling algorithm.

Additionally, the Comparison Engine is connected to this layer through a ComparisonService interface. The Comparison Engine receives scheduling results and analyzes the performance of different algorithms by comparing metrics such as total seek time. This enables the system to determine which algorithm performs more efficiently under given conditions.

The interface between IS Scheduler and Comparison Engine is called SchedulingResultService. It is used to transfer the scheduling results from the selected algorithm to the Comparison Engine.

After an algorithm calculates the seek time and sequence, the results are provided through this interface. The Comparison Engine receives these results and compares the performance of different algorithms.

This interface ensures loose coupling and keeps the scheduling and comparison components independent and modular.

2.4-Is Scheduler and Disk Scheduling Algorithms

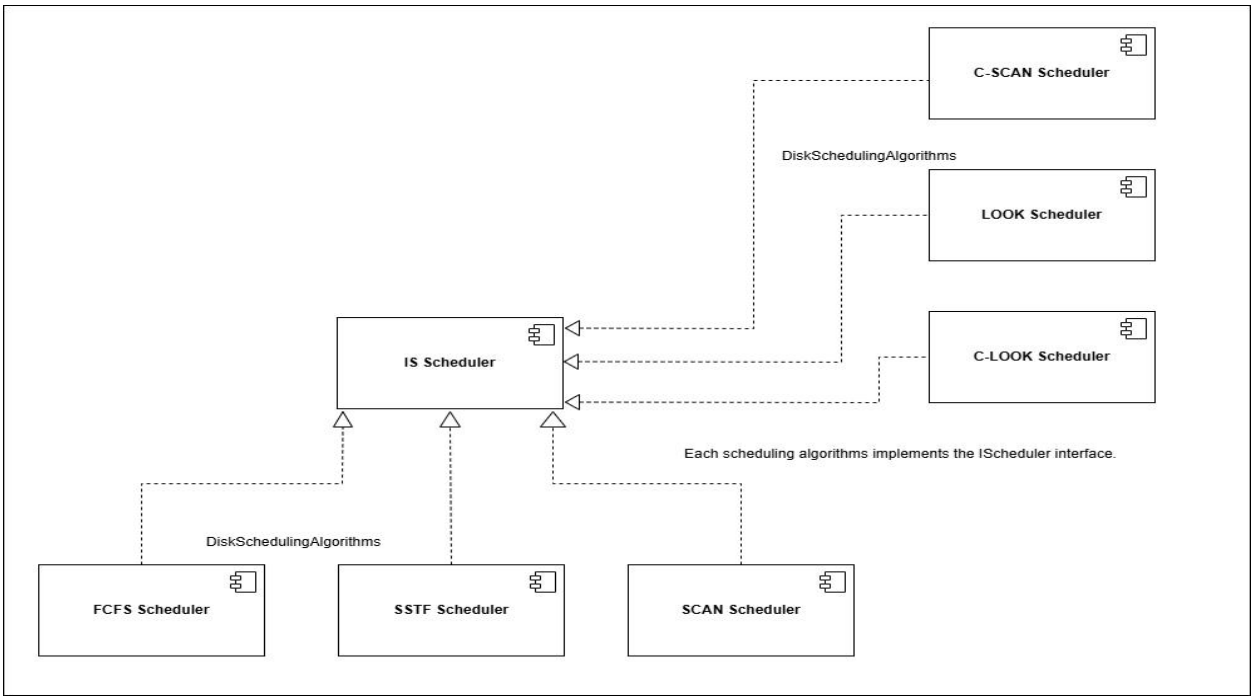

Figure: 2.4 -Is Scheduler and Algorithms

Explanation:

This part of the component diagram represents the implementation layer of the Disk Scheduling Simulator. It includes multiple disk scheduling algorithms such as FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK.

All these algorithms implement the IScheduler interface. The IScheduler acts as a common contract that defines the standard scheduling operations. Because of this, every algorithm must follow the same structure and provide the required scheduling functions.

The Simulation Controller does not directly depend on any specific algorithm. Instead, it communicates through the IScheduler interface. This design allows the system to switch between different algorithms easily without changing the controller logic.

This architecture follows object-oriented principles such as polymorphism and abstraction. It makes the system modular, flexible, and easy to extend.

The dashed lines represent dependency and interface relationships, showing that the scheduling algorithms communicate through the IScheduler interface instead of direct connections.

The hollow triangle symbol represents interface implementation (realization), indicating that each scheduling algorithm implements the IScheduler interface.

3.FINAL COMPONENT DIAGRAM OF DISK SCHEDULING SYSTEM

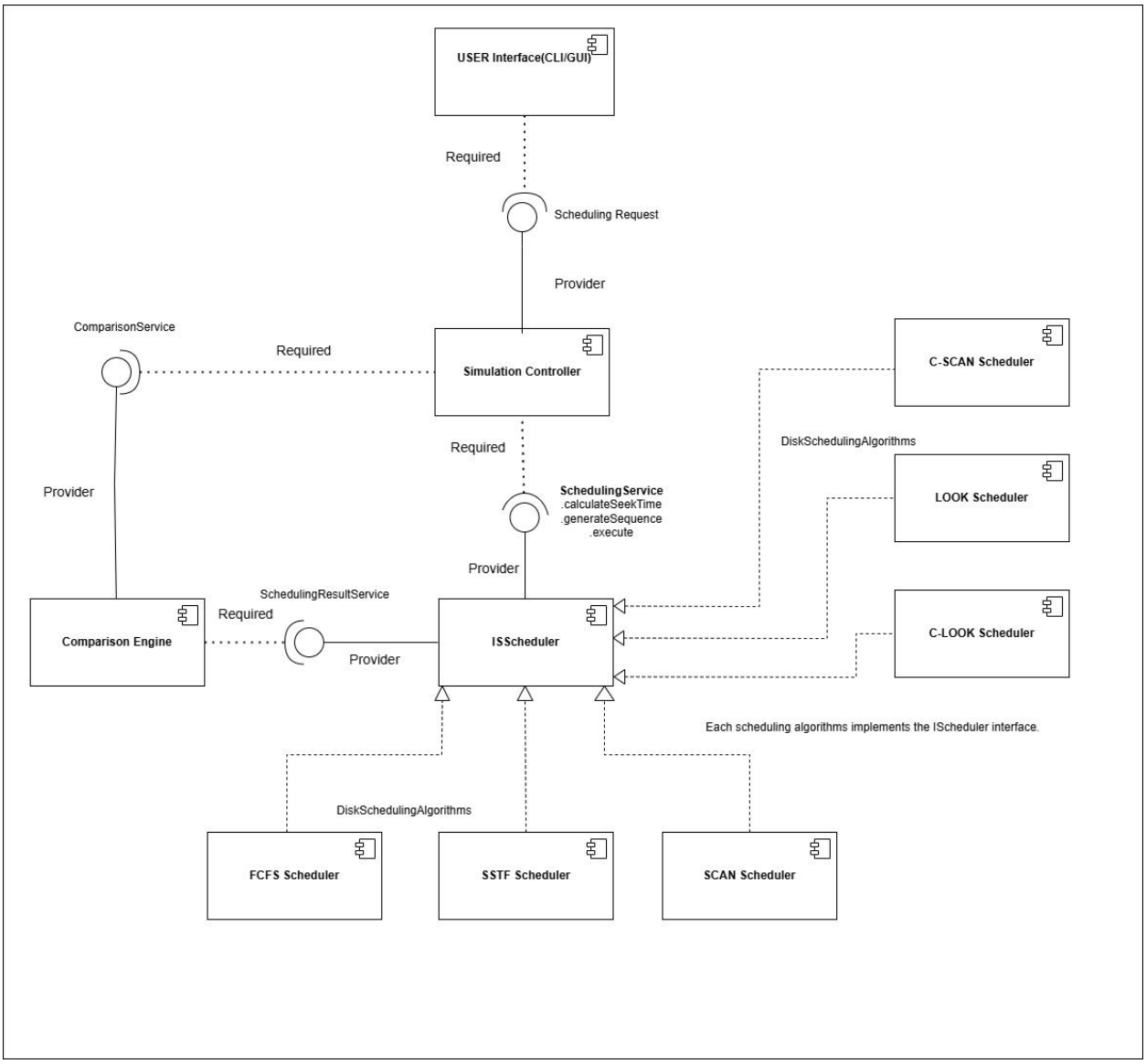

Figure: 3.1 Component diagram of disk scheduling system

Explanation:

This component diagram shows the overall architecture of the Disk Scheduling Simulator and how all components work together.

The process starts with the User Interface (CLI/GUI). The user enters disk requests, initial head position, and selects a scheduling algorithm. The request is then sent to the Simulation Controller.

The Simulation Controller manages the whole scheduling process. It communicates with the SchedulingService, which defines the main functions such as calculating seek time, generating the seek sequence, and executing the selected algorithm.

All disk scheduling algorithms like FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK implement the IScheduler interface. This means every algorithm follows the same structure. Because of this design, the controller can switch between algorithms easily without changing the main logic.

After the algorithm calculates the results, the data is sent to the Comparison Engine through the SchedulingResultService interface. Comparison Engine compares the performance of different algorithms based on total seek time and determines which algorithm performs better.

Overall, this architecture is modular, flexible, and easy to maintain. It allows new scheduling algorithms to be added without affecting the existing system.